

DIGITAL FORENSICS

COMPARATIVE STUDY ON MYSQL AND NOSQL INJECTION DETECTION

TEAM:

SHIVANI J - PES₁UG₂₃CS₅₄₉

SHREEVARSHA S - PES₁UG₂₃CS₅₅₅

INTRODUCTION

Digital forensics involves the investigation of digital devices to uncover and interpret electronic data. It plays a crucial role in cybersecurity, legal investigations, and corporate compliance. With the increasing prevalence of cybercrime, the need for skilled digital forensics professionals has never been greater.

OBJECTIVE

MySQL (SQL Injection Detection): Used forensic-based detection in a Node.js app with MySQL by:

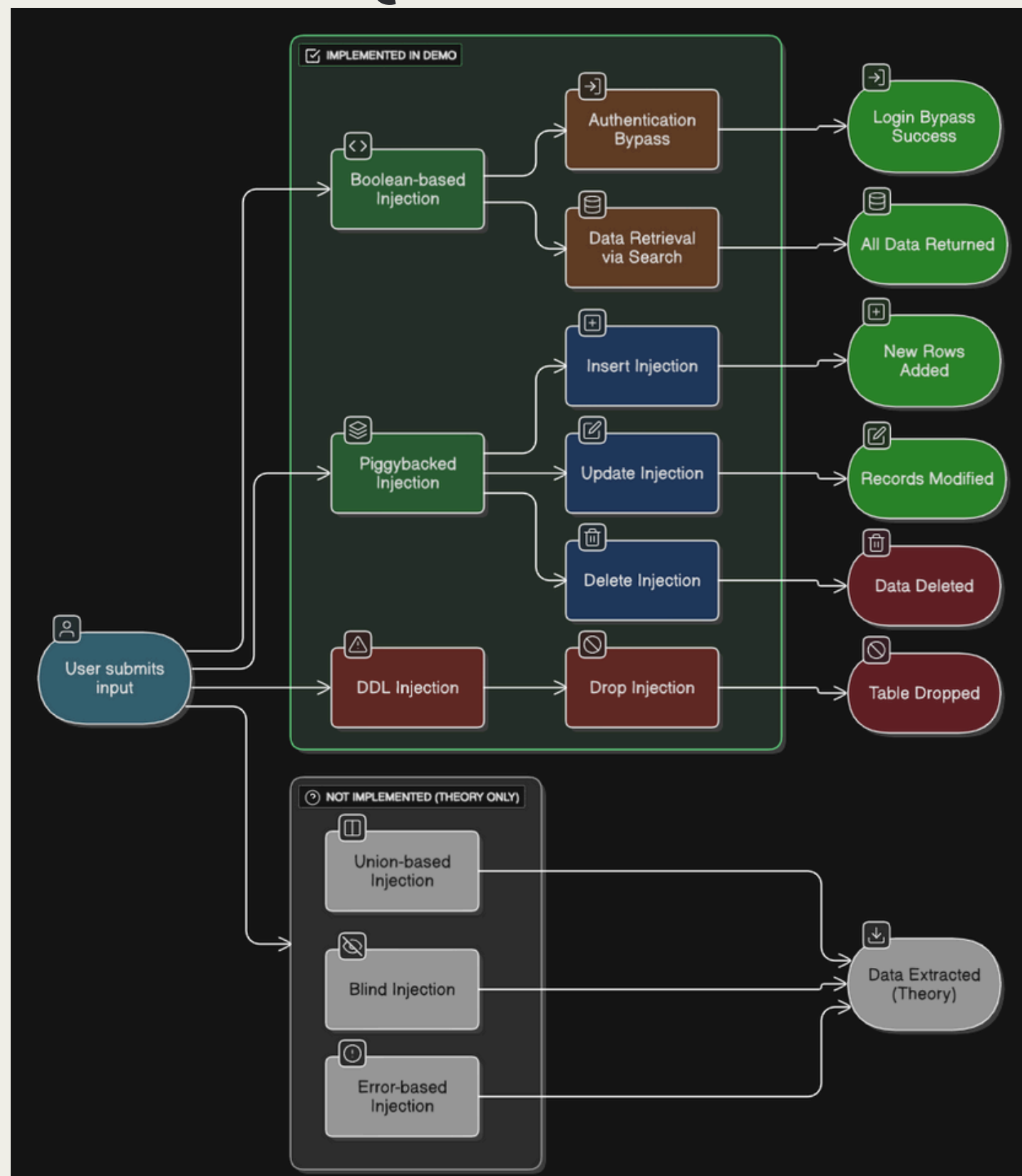
- Creating secure (prepared) and vulnerable routes
- Simulating attacks via SQLMap
- Logging inputs, IPs, and query traces
- Detecting injections using heuristic rules and a lightweight ML model

MongoDB (NoSQL Injection Detection): Used forensic-based detection in a Node.js app with NoSQL by:

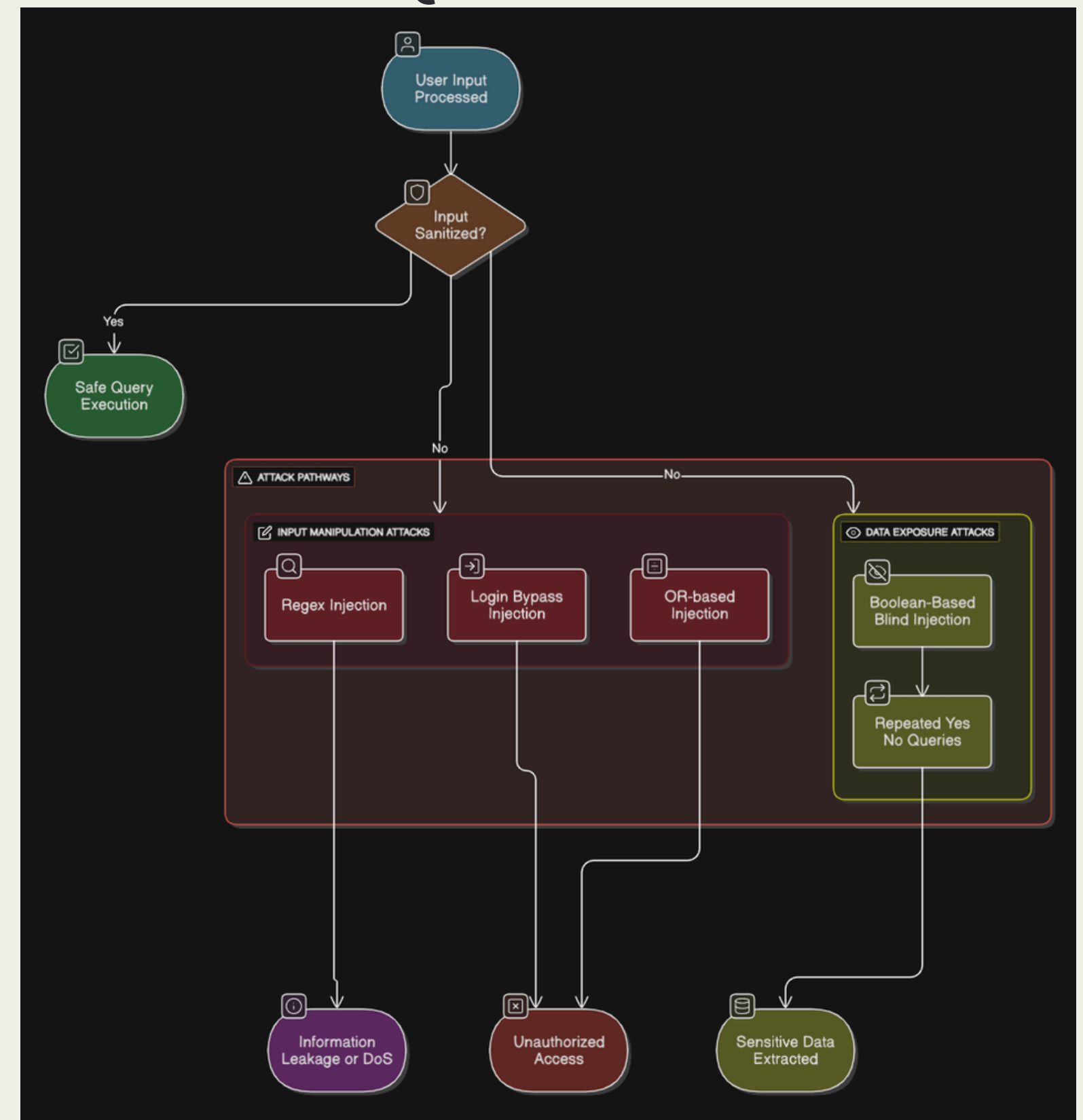
- Building both insecure and secure endpoints
- Testing attack vectors like \$ne, \$or, and regex payloads
- Logging payloads and query behavior
- Detecting anomalies using heuristics and a custom ML model
- Evaluating hybrid detection effectiveness (heuristic + ML)

MYSQL AND NOSQL TYPES

MYSQL AND TYPES



NOSQL AND TYPES



HOW MYSQL INJECTION IS PERFORMED

MySQL Injection – Login Bypass

- Occurs when user input is inserted directly into SQL queries.
- Attacker modifies input to alter query logic.
- Can reveal data, bypass login, or even delete tables.
- Very common in poorly secured web apps.

Eg:

Input: ' OR '1'='1

Query: SELECT * FROM users WHERE username = " OR '1'='1' AND password = "

HOW NOSQL INJECTION IS PERFORMED

NoSQL Injection – Login Bypass

- Happens when raw user input is directly inserted into NoSQL queries (e.g., MongoDB).
- Attackers inject JSON operators to manipulate query behavior.
- Can bypass authentication or access unauthorized data.

Eg:

Input: `{"username": {"$ne": null}}`

Query: `db.users.find({ "username": { "$ne": null } , "password": "..."} )`

WHAT IS HEURISTICS

- Simple rule-based method to detect suspicious activity
- Works using logic and known attack patterns
- Common in detecting things like ' OR 1=1 -- in SQL
- Fast and easy to implement

EX:

MYSQL:

Input: ' OR 1=1 --

Query: *SELECT * FROM users WHERE username = " OR 1=1 -- ' AND password = ";*

NOSQL(MONGODB):

Input: { "\$ne": null }

Query: *db.users.findOne({ username: { \$ne: null }, password: { \$ne: null } });*

MACHINE LEARNING FOR INJECTION DETECTION

Why Machine Learning?

- Heuristic rules may miss obfuscated or zero-day attacks
- ML generalizes patterns from known malicious inputs
- Helps reduce false positives in SQL/NoSQL detection

Our Approach

- Model Used: Logistic Regression
- Dataset: Labeled inputs (benign vs. injection payloads)
- Features Extracted:
 - Special characters (' , --, \$ne, etc.)
 - SQL/JS keywords (SELECT, DROP, eval)
 - Input length, entropy, character ratios

MACHINE LEARNING FOR INJECTION DETECTION

Integration into Web App

- Inputs pre-processed and passed to Logistic Regression model
- Malicious inputs flagged, logged (IP, payload, query)
- Real-time inference with low latency

Results

- Detected most injection patterns not caught by heuristics
- Reduced false positives on user-submitted content
- Maintained usability and performance

Future Improvements

- Retrain with more diverse payloads (including obfuscated types)
- Add NLP-based feature extraction (token patterns, embeddings)
- Compare Logistic Regression with more complex models (e.g., Random Forest, SVM)

CONCLUSION

- At the end we can conclude that, SQL and NoSQL databases are both prone to injection attacks.
- Heuristics give quick detection but may miss advanced attacks
- Machine Learning improves accuracy and reduces false alarms
- A combined Heuristics + ML approach ensures stronger security
- Continuous improvement with larger datasets and advanced models can further enhance protection

Thank you!
